

Penguins Database - Clusters

Diplomado de métodos matemáticos y estadísticos para la Ciencia de Datos

Equipo 7

Integrantes:

Fajardo Zavala Yendali Viridiana
González Rivas José Rafael
Maldonado Zamora Silvia Viridiana
Olivera Flores Jonathan
Valencia Quiroz Luis Roberto

2026-01-16

Descripción

El dataset penguins.csv contiene mediciones biométricas de pingüinos recolectadas en las Islas Palmer (Antártida). Contiene 344 registros y 8 variables. El objetivo de este trabajo es explorar, describir y reducir la dimensionalidad de las variables numéricas para identificar patrones y relaciones entre individuos, y evaluar si estas mediciones permiten distinguir especies mediante técnicas como PCA, t-SNE y UMAP.

Variables

El dataset incluye variables categóricas, por ejemplo:

1. species: Categórica. La especie del pingüino (Adelie, Chinstrap o Gentoo).
2. island: Categórica. La isla de observación (Torgersen, Biscoe o Dream).

Además, incluye variables numéricas relacionadas con rasgos físicos, como:

3. bill_length_mm: Numérica (float). Longitud del pico en milímetros.
4. bill_depth_mm: Numérica (float). Profundidad del pico en milímetros.
5. flipper_length_mm: Numérica (float). Longitud de la aleta en milímetros.
6. body_mass_g: Numérica (float). Masa corporal en gramos.
7. sex: Categórica. El sexo del pingüino (male o female).
8. year: Numérica (integer). El año de la observación.

Estas variables permiten comparar individuos, analizar distribuciones y explorar qué mediciones aportan más información para separar especies

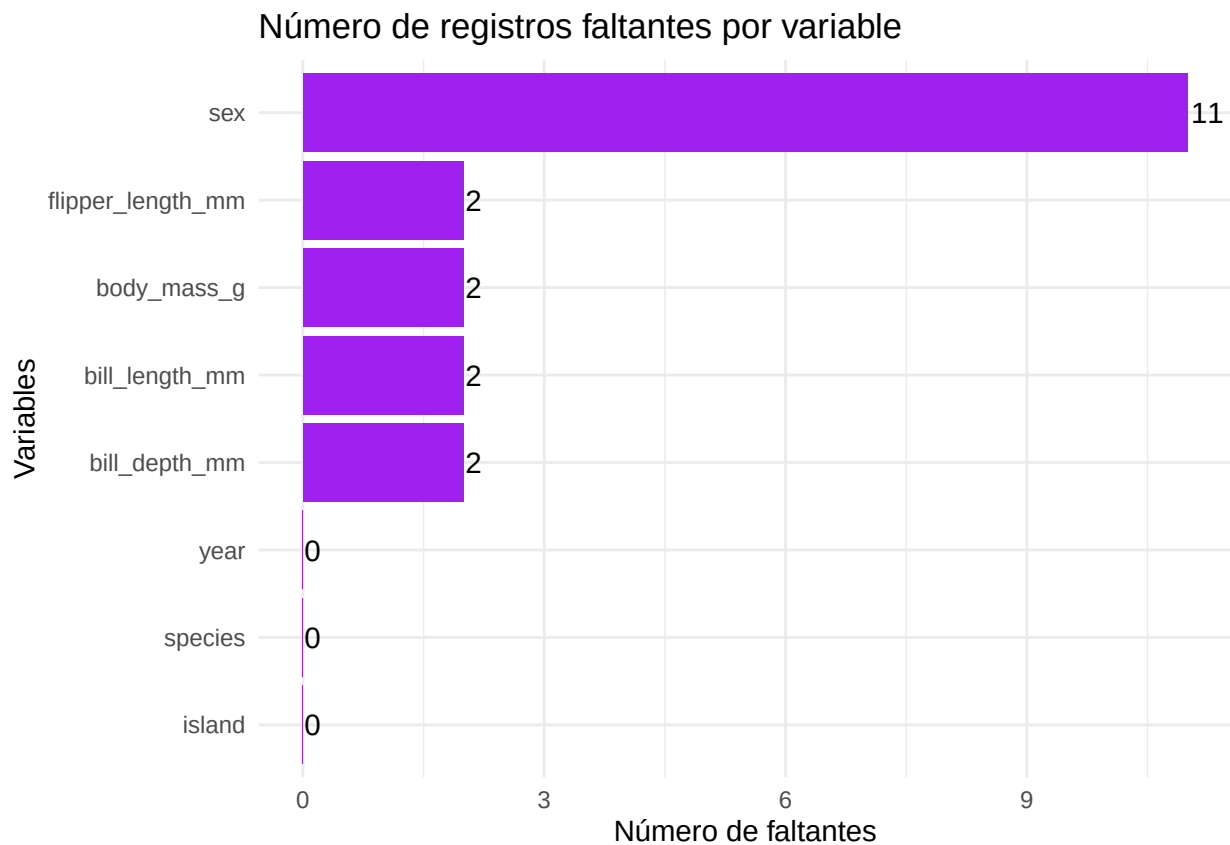
Explorando el dataset

Table 1: Dimensión del conjunto de datos

Métrica	Valor
Número de observaciones	344
Número de variables	8

Table 2: Primeras filas del conjunto de datos

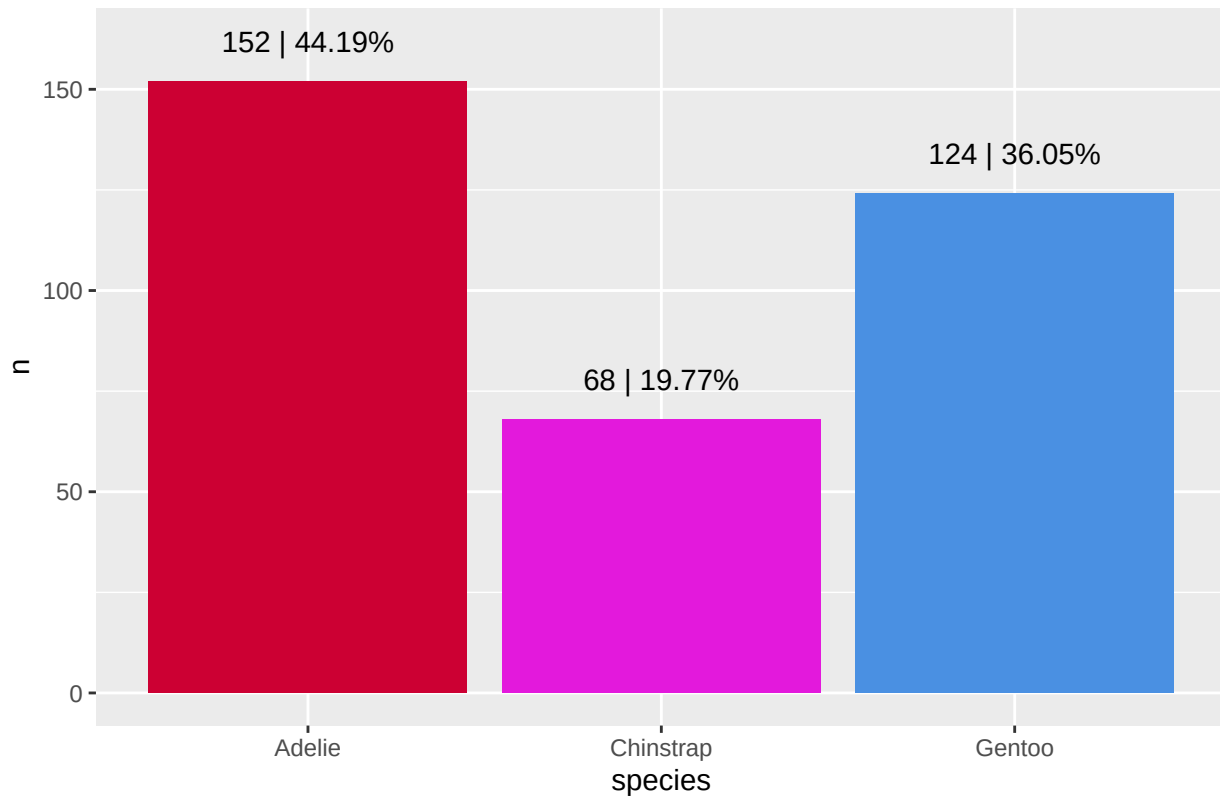
species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex	year
Adelie	Torgersen	39.1	18.7	181	3750	male	2007
Adelie	Torgersen	39.5	17.4	186	3800	female	2007
Adelie	Torgersen	40.3	18.0	195	3250	female	2007
Adelie	Torgersen	NA	NA	NA	NA	NA	2007
Adelie	Torgersen	36.7	19.3	193	3450	female	2007
Adelie	Torgersen	39.3	20.6	190	3650	male	2007



No se identificaron registros duplicados en el conjunto de datos.

Se transforma species a factor para usarla como variable de clasificación/etiqueta en las gráficas comp

Porcentaje de individuos por especie



Preparación de los datos para clustering

```
df1 <- df %>%
  select(where(is.numeric)) %>%
  select(-year) %>%
  drop_na() %>%
  scale() %>%
  as.data.frame()
```

```
summary(df1)
```

```
##  bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g
##  Min.   :-2.16535  Min.   :-2.05144  Min.   :-2.0563  Min.   :-1.8726
##  1st Qu.: -0.86031  1st Qu.: -0.78548  1st Qu.: -0.7762  1st Qu.: -0.8127
##  Median :  0.09672  Median :  0.07537  Median : -0.2784  Median : -0.1892
##  Mean   :  0.00000  Mean   :  0.00000  Mean   :  0.0000  Mean   :  0.0000
##  3rd Qu.:  0.83854  3rd Qu.:  0.78430  3rd Qu.:  0.8594  3rd Qu.:  0.6836
##  Max.   :  2.87166  Max.   :  2.20217  Max.   :  2.1395  Max.   :  2.6164
```

```
head(df1)
```

```
##  bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g
##  1    -0.8832047    0.7843001    -1.4162715   -0.5633167
##  2    -0.8099390    0.1260033    -1.0606961   -0.5009690
##  3    -0.6634077    0.4298326    -0.4206603   -1.1867934
##  4    -1.3227986    1.0881294    -0.5628905   -0.9374027
```

```
## 5      -0.8465718      1.7464261      -0.7762357  -0.6880121
## 6      -0.9198375      0.3285561      -1.4162715  -0.7191859
```

```
# Calculando distancias (distancia euclidean)
```

```
dist_df <- dist(df1, method = "euclidean")
```

```
hc_ave    <- hclust(dist_df, method = "average")
hc_single <- hclust(dist_df, method = "single")
hc_comp   <- hclust(dist_df, method = "complete")
hc_ward   <- hclust(dist_df, method = "ward.D")
hc_ward2  <- hclust(dist_df, method = "ward.D2")
```

```
###Distancia cophenetic
```

```
d_ave     <- cophenetic(hc_ave)
d_single  <- cophenetic(hc_single)
d_comp    <- cophenetic(hc_comp)
d_ward    <- cophenetic(hc_ward)
d_ward2   <- cophenetic(hc_ward2)
```

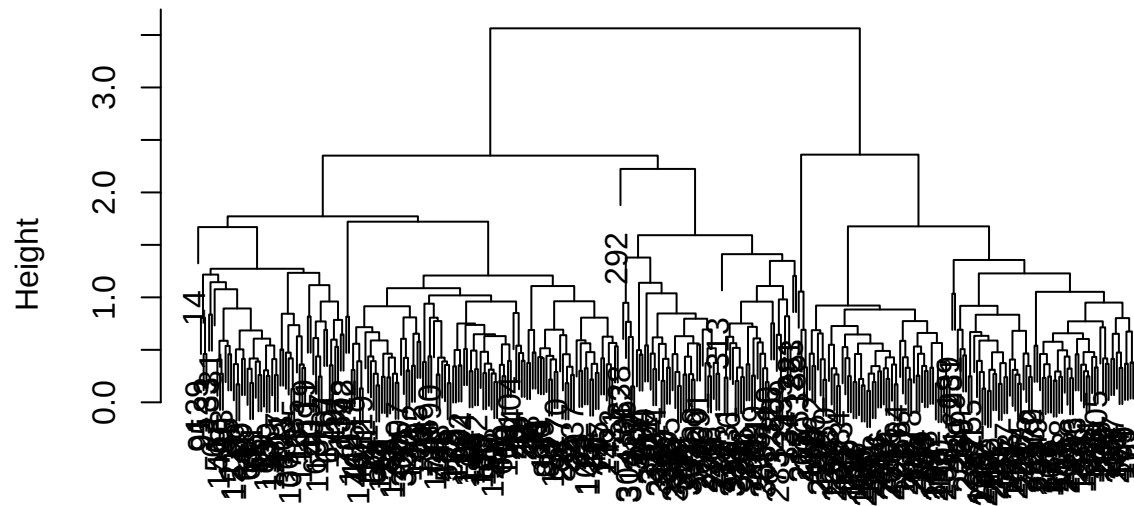
```
cbind(
  average = cor(dist_df, d_ave),
  single  = cor(dist_df, d_single),
  complete = cor(dist_df, d_comp),
  ward    = cor(dist_df, d_ward),
  ward2   = cor(dist_df, d_ward2)
)
```

```
##          average    single complete      ward      ward2
## [1,] 0.8447094 0.8131742 0.8281833 0.8188395 0.8346438
```

```
# Dendrogramas
```

```
plot(hc_ave)
```

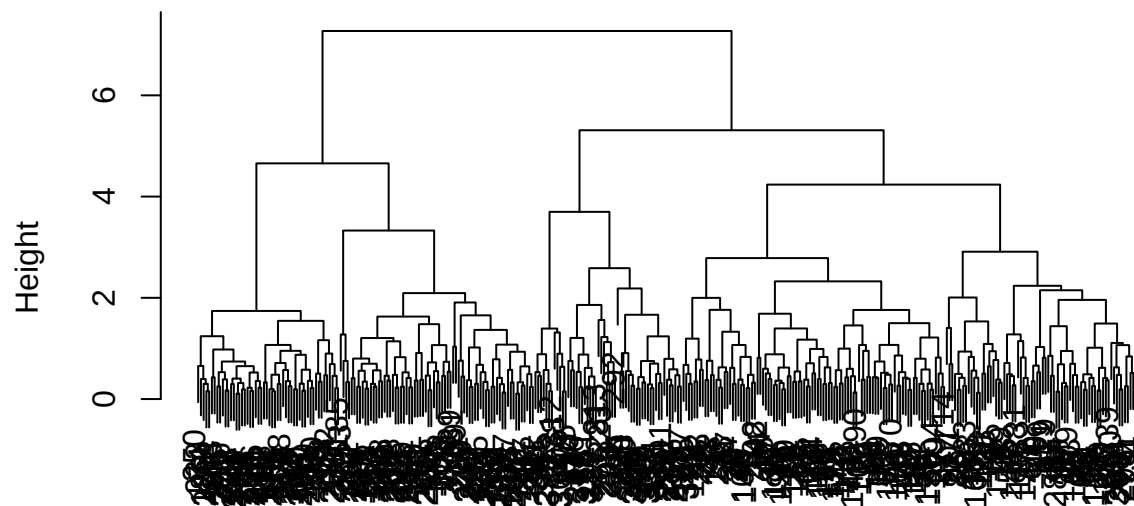
Cluster Dendrogram



```
dist_df
hclust (*, "average")
```

```
plot(hc_comp)
```

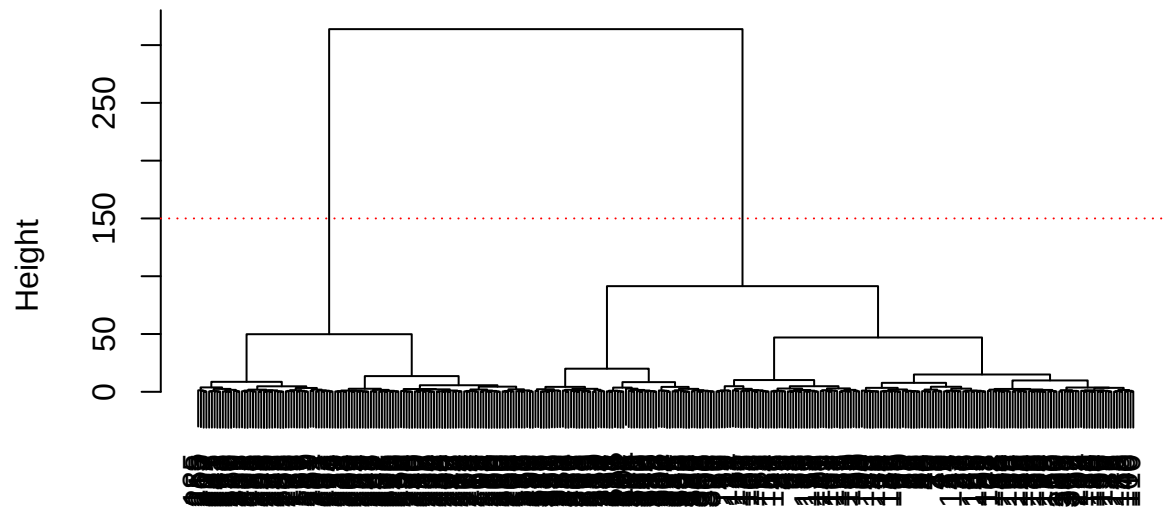
Cluster Dendrogram



```
dist_df
hclust (*, "complete")
```

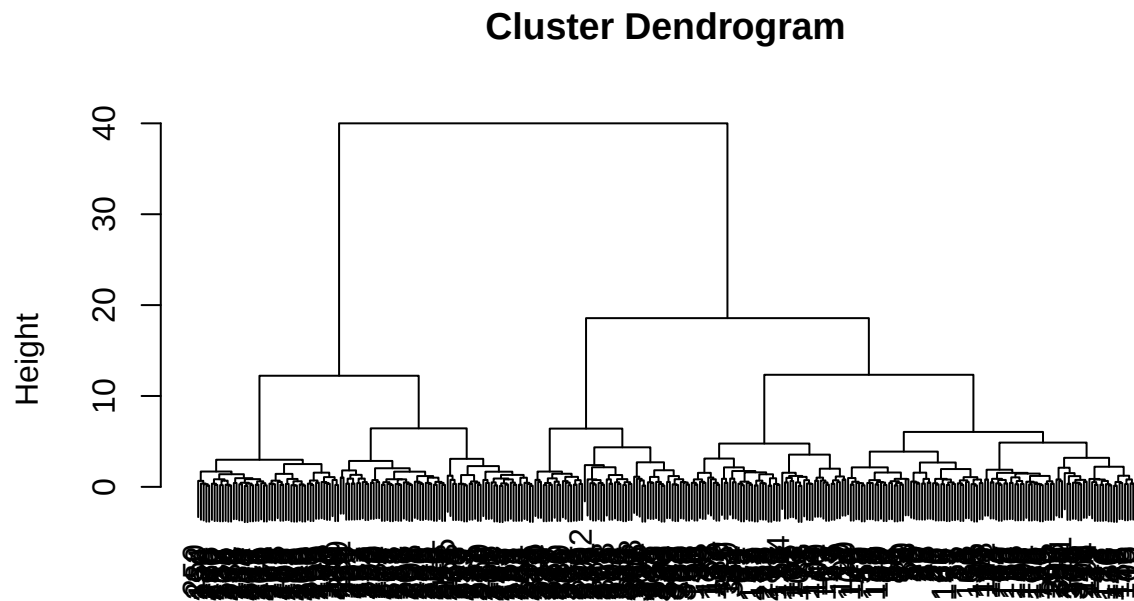
```
plot(hc_ward)
abline(h = 150, lty = 3, col = "red")
```

Cluster Dendrogram



dist_df
hclust (*, "ward.D")

```
plot(hc_ward2)
abline(h = 45, lty = 3, col = "red")
```

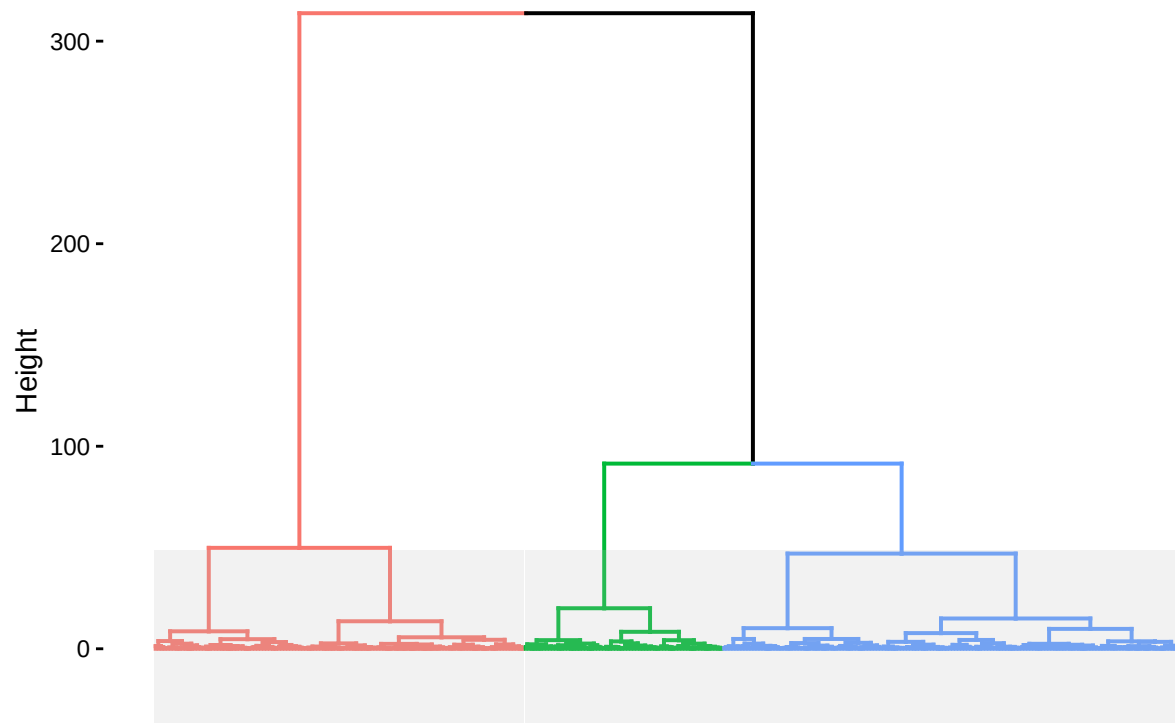


```
dist_df
hclust (*, "ward.D2")
```

```
fviz_dend(
  hc_ward,
  cex = 0.65,
  k = 3,
  color_labels_by_k = FALSE,
  rect = TRUE,
  rect_fill = TRUE,
  show_labels = FALSE,
  main = "Cluster: Liga ward dendrograma (2 grupos)"
)
```

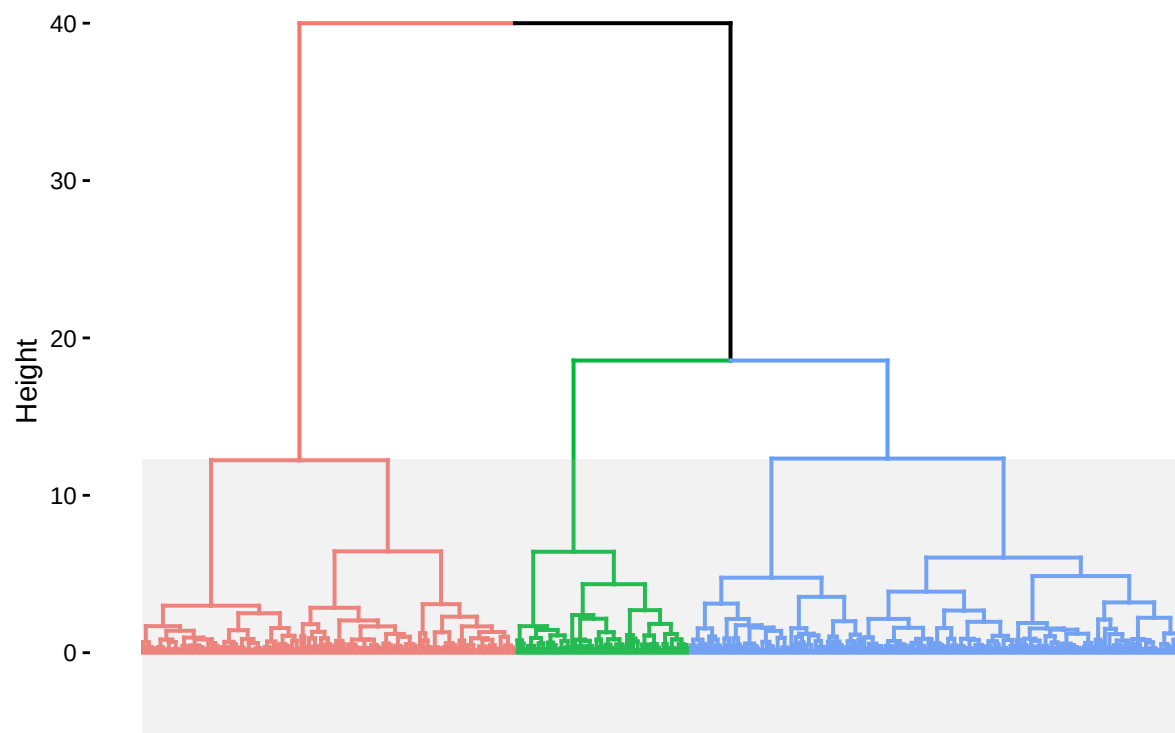
```
## Registered S3 methods overwritten by 'dendextend':
##   method      from
##   rev.hclust   vegan
##   text.pvclust pvclust
```

Cluster: Liga ward dendrograma (2 grupos)



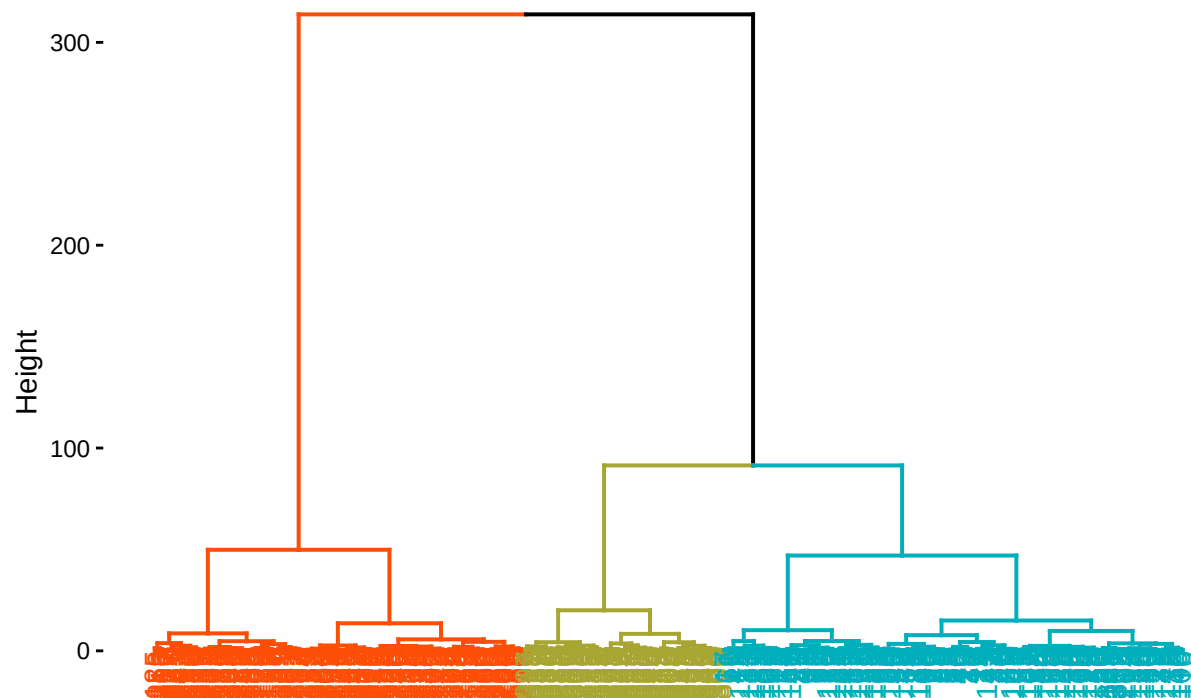
```
fviz_dend(  
  hc_ward2,  
  cex = 0.65,  
  k = 3,  
  color_labels_by_k = FALSE,  
  rect = TRUE,  
  rect_fill = TRUE,  
  show_labels = FALSE,  
  main = "Cluster: Liga ward2 dendrograma (2 grupos)"  
)
```


Cluster: Liga ward2 dendrograma (2 grupos)

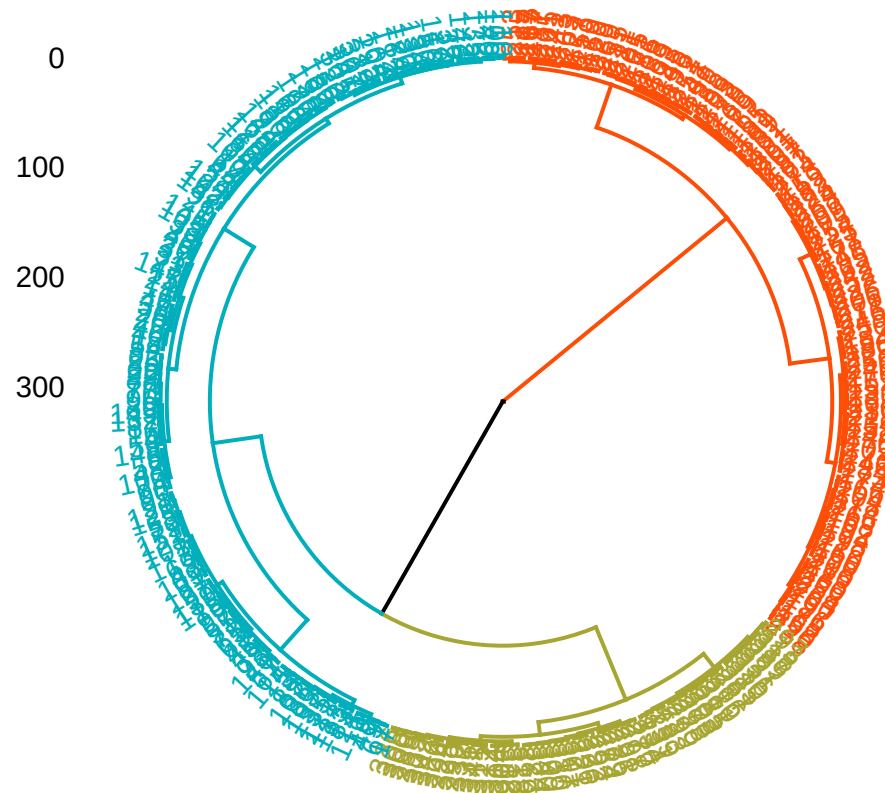


```
fviz_dend(hc_ward, cex = 0.8, k = 3, k_colors = c("#FC4E07", "#a8a632", "#00AFBB"), type = "rectangle")
```

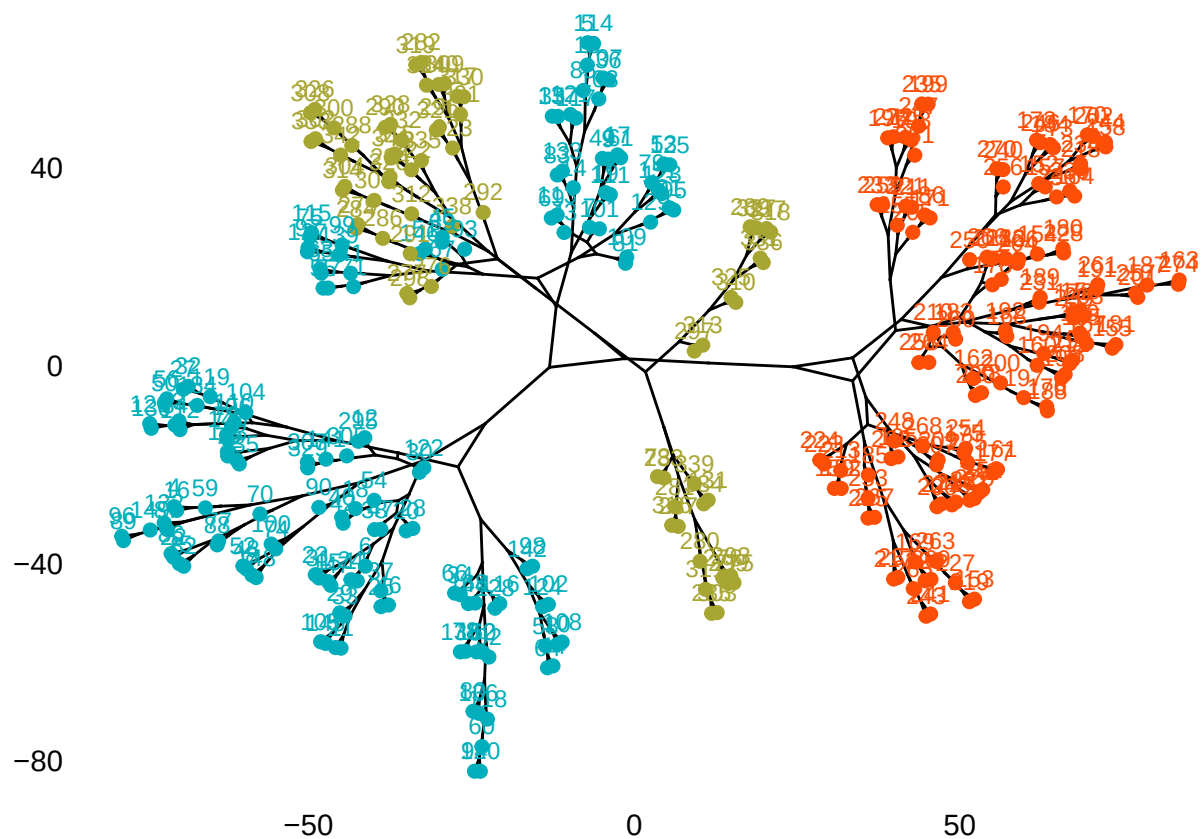
Cluster Dendrogram



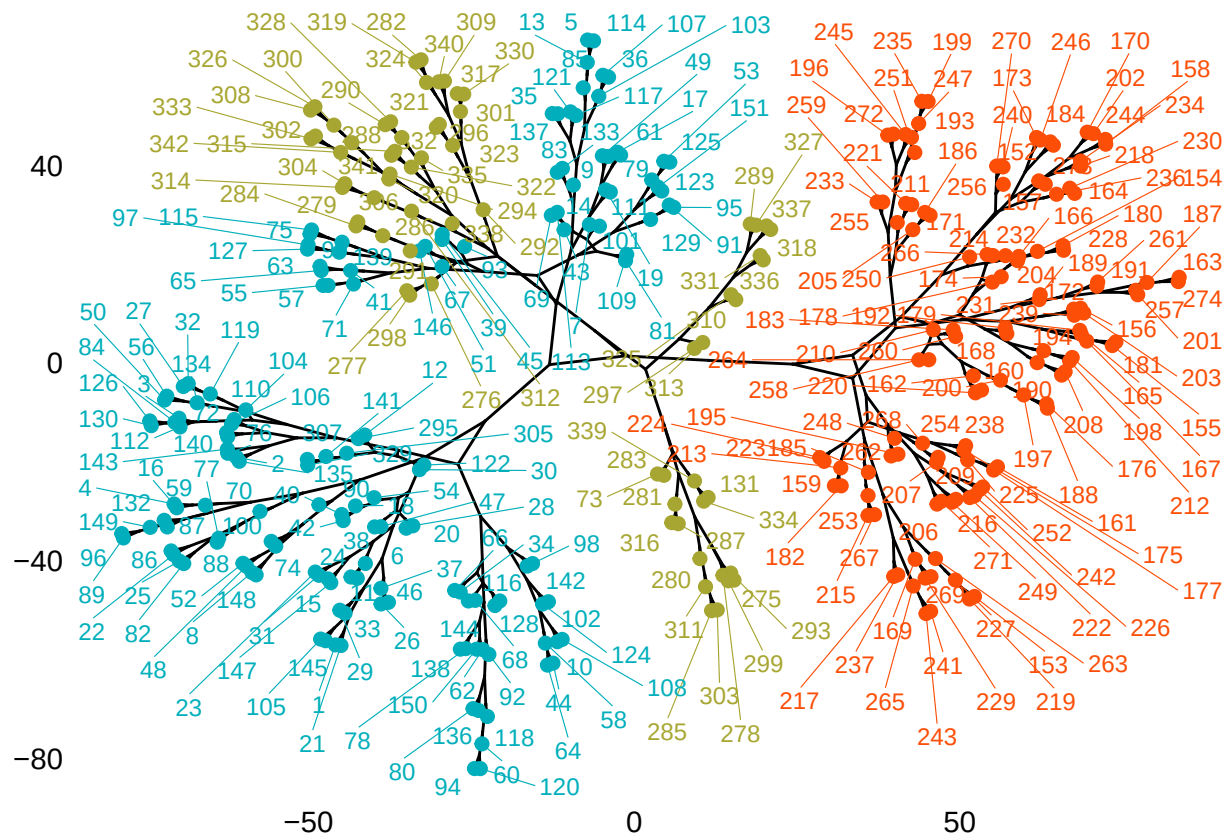
```
fviz_dend(hc_ward, cex = 0.8, k = 3, k_colors = c("#FC4E07", "#a8a632", "#00AFBB"), type = "circular")
```



```
fviz_dend(hc_ward, cex = 0.8, k = 3, k_colors = c("#FC4E07", "#a8a632", "#00AFBB"), type = "phylogenetic")
```



```
fviz_dend(hc_ward, cex = 0.8, k = 3, k_colors = c("#FC4E07", "#a8a632", "#00AFBB"), type = "phylogenetic", r
```



```
###Con otro metodo aglomerativo: agnes
```

```
df1_df <- as.data.frame(df1)
```

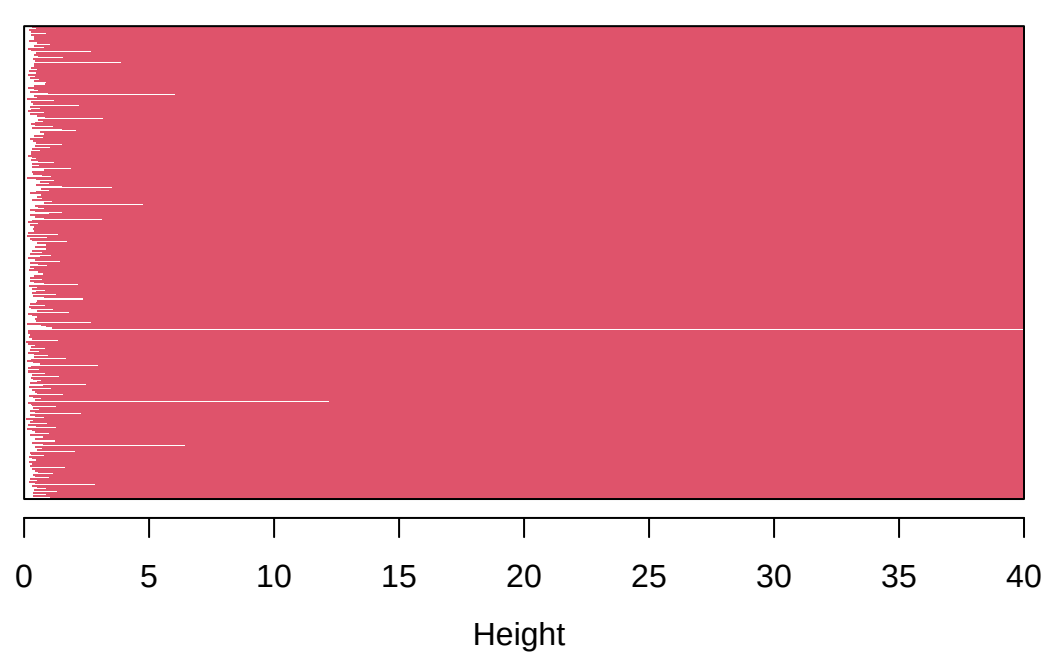
```
ac_metric <- list(
  complete_ac = agnes(df1, metric = "euclidean", method = "complete")$ac,
  average_ac = agnes(df1, metric = "euclidean", method = "average")$ac,
  single_ac = agnes(df1, metric = "euclidean", method = "single")$ac,
  ward_ac = agnes(df1, metric = "euclidean", method = "ward")$ac
)
```

```
ac_metric
```

```
## $complete_ac
## [1] 0.9460713
##
## $average_ac
## [1] 0.8924129
##
## $single_ac
## [1] 0.7687883
##
## $ward_ac
## [1] 0.990277
```

```
plot(agnes(df1, metric = "euclidean", method = "ward"))
```

Banner of `agnes(x = df1, metric = "euclidean", method = "ward")` Dendrogram of `agnes(x = df1, metric = "euclidean", method = "ward")`

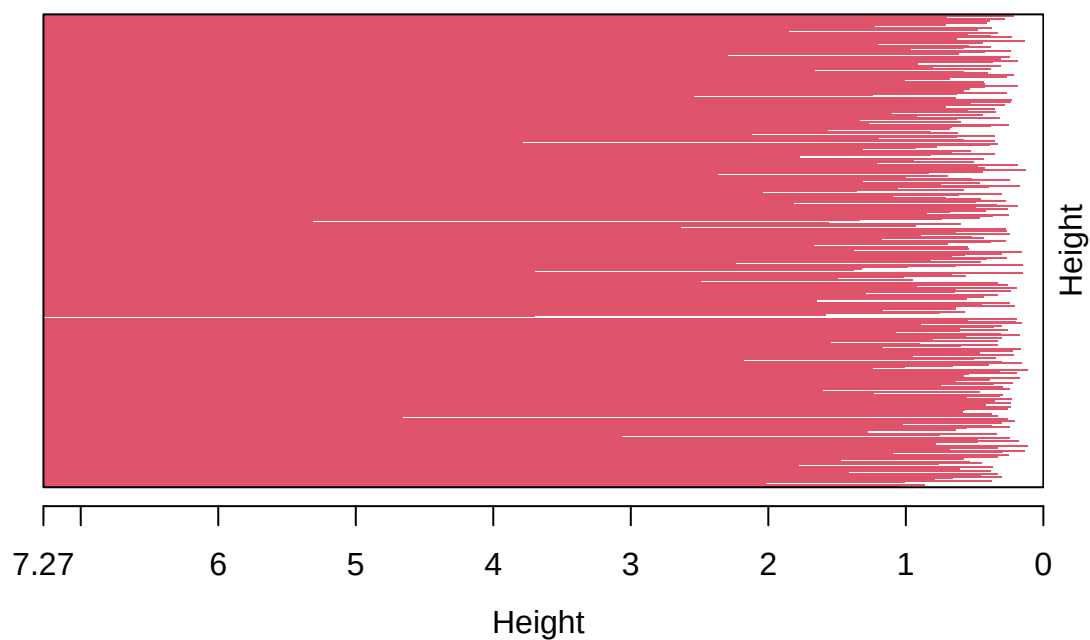


Agglomerative Coefficient = 0.99

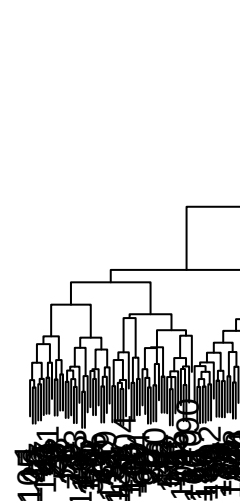
###Con un metodo divisivo: diana

```
plot(diana(df1, metric = "euclidean"))
```

Banner of `diana(x = df1, metric = "euclidean")`



Dendrogram



Divisive Coefficient = 0.94

```
###Con tidymodels
###Sin declarar liga

hclust_spec <- hier_clust() %>%
  set_engine("stats")

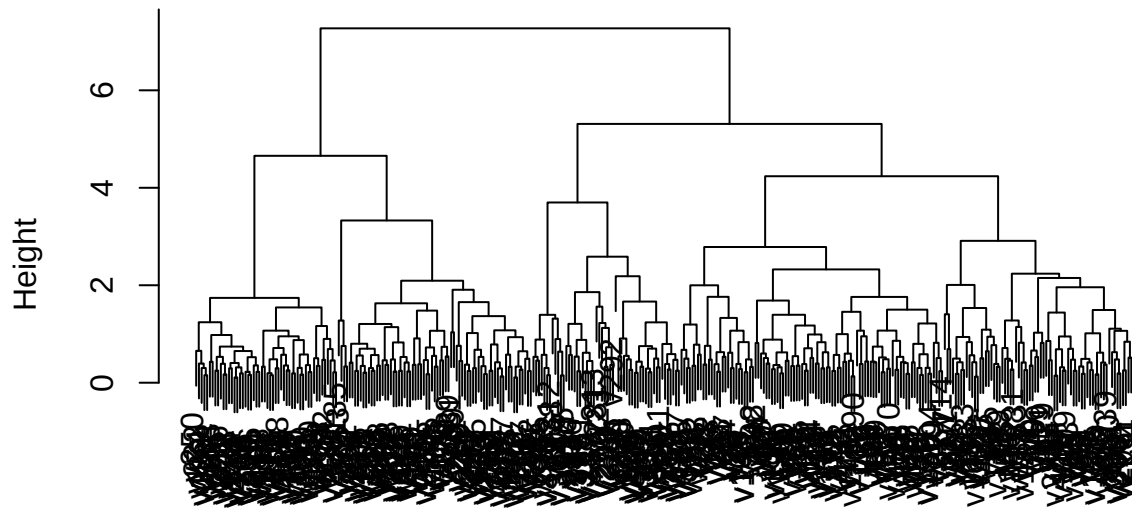
hclust_fit1 <- tidyclust::fit(hclust_spec, ~., df1)

hclust_fit1 %>%
  extract_centroids(num_clusters = 2)%>%print(Inf)
```

```
## # A tibble: 2 x 5
##   .cluster bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <fct>      <dbl>      <dbl>      <dbl>      <dbl>
## 1 Cluster_1    -0.369      0.617     -0.650     -0.612
## 2 Cluster_2     0.656     -1.10      1.16      1.09
```

```
hclust_fit1$fit %>% plot()
```

Cluster Dendrogram



```
stats::as.dist(dmat)
stats::hclust(*, "complete")
```

```
###Declarando ligas
```

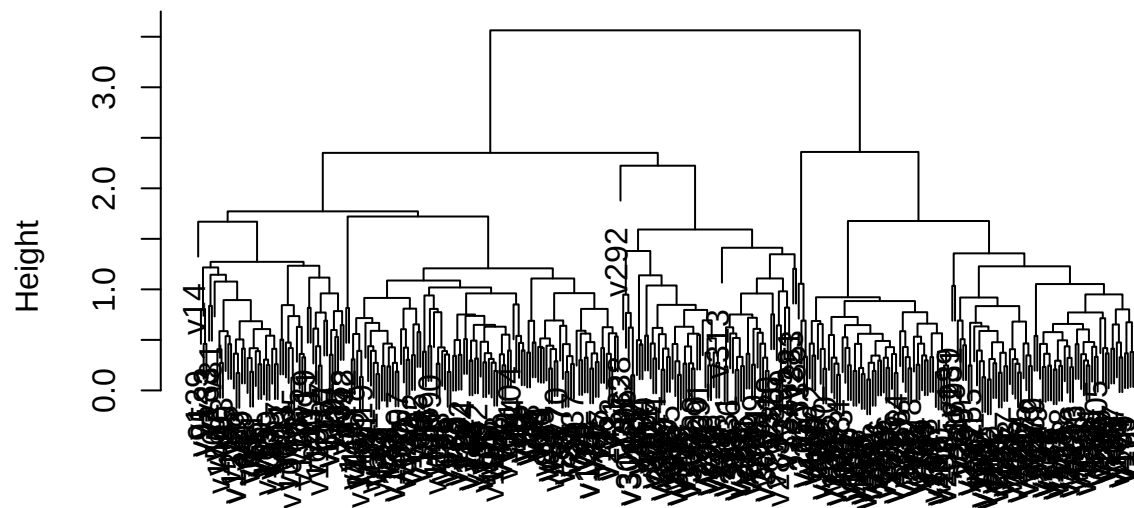
```
res_hclust_complete <- hier_clust(linkage_method = "complete") %>%
  tidyclust::fit(~., data = df1)
```

```
res_hclust_complete$fit %>% plot()
```

```
res_hclust_average <- hier_clust(linkage_method = "average") %>%
  tidyclust::fit(~., data = df1)
```

```
res_hclust_average$fit %>% plot()
```

Cluster Dendrogram

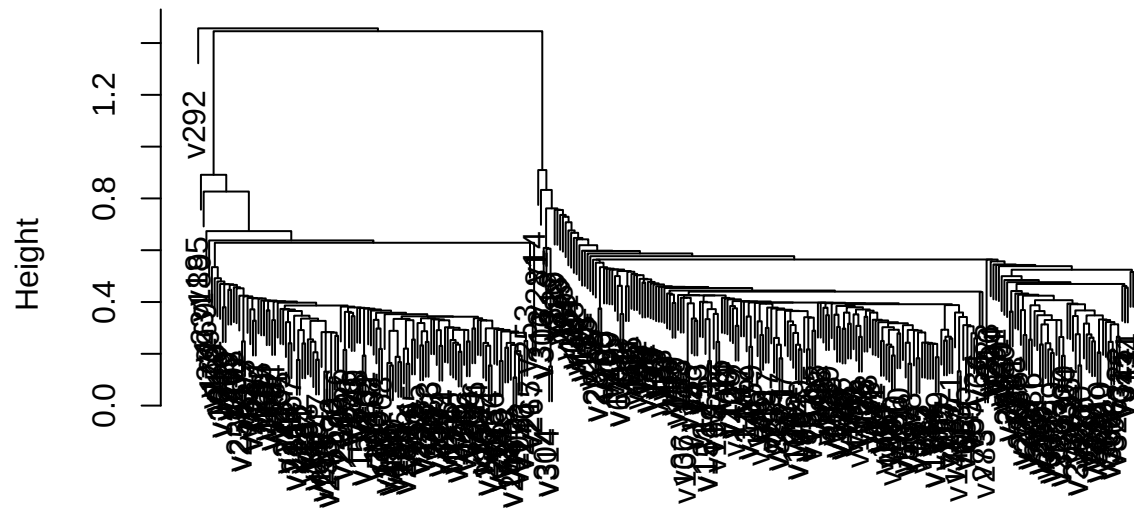


```
stats::as.dist(dmat)
stats::hclust (*, "average")
```

```
res_hclust_single <- hier_clust(linkage_method = "single") %>%
  tidyclust::fit(~., data = df1)

res_hclust_single$fit %>% plot()
```


Cluster Dendrogram

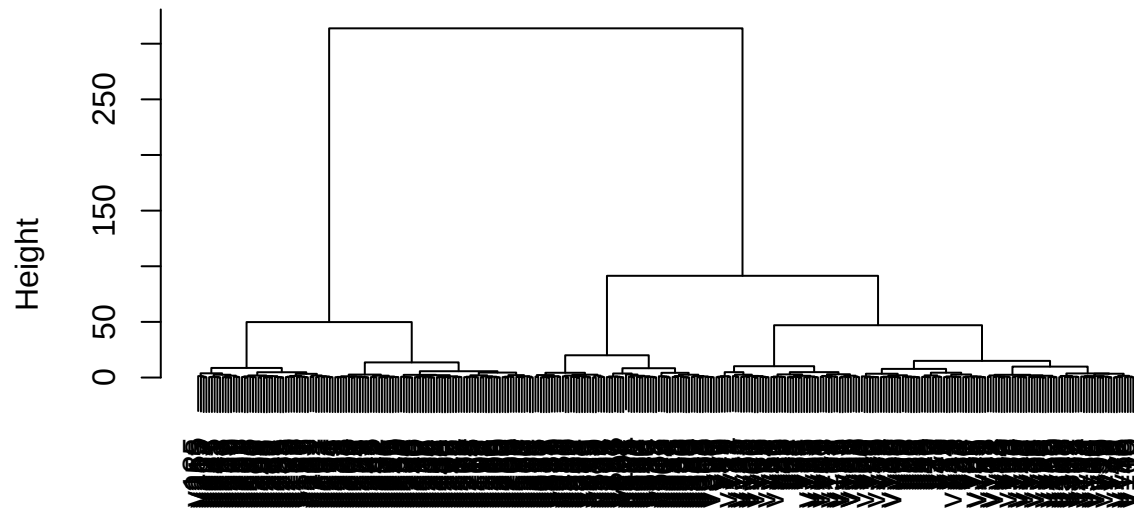


```
stats::as.dist(dmat)
stats::hclust (*, "single")
```

```
res_hclust_ward <- hier_clust(linkage_method = "ward.D") %>%
  tidyclust::fit(~., data = df1)

res_hclust_ward$fit %>% plot()
```

Cluster Dendrogram

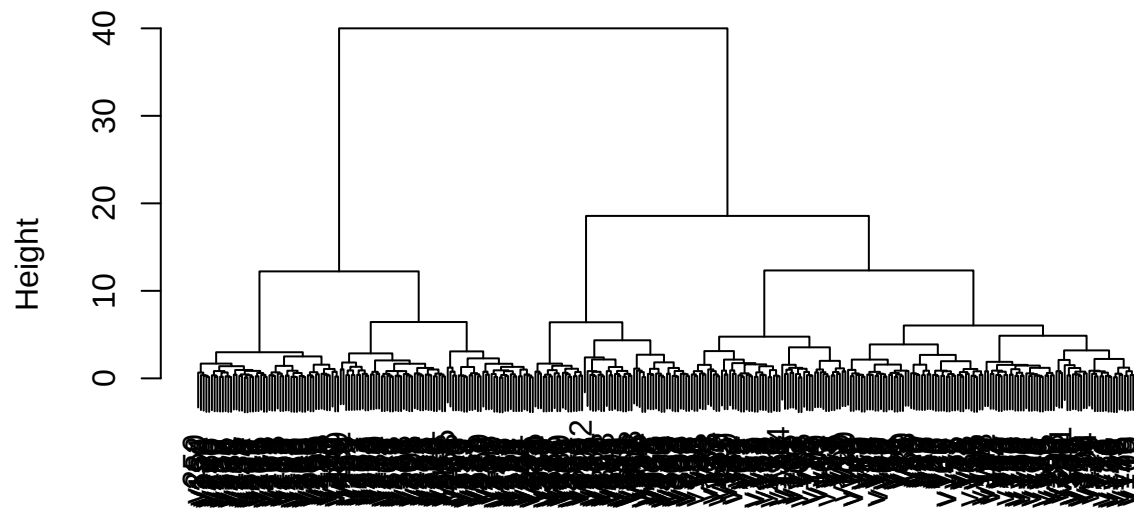


```
stats::as.dist(dmat)
stats::hclust (*, "ward.D")
```

```
res_hclust_ward2 <- hier_clust(linkage_method = "ward.D2") %>%
  tidyclust::fit(~., data = df1)

res_hclust_ward2$fit %>% plot()
```

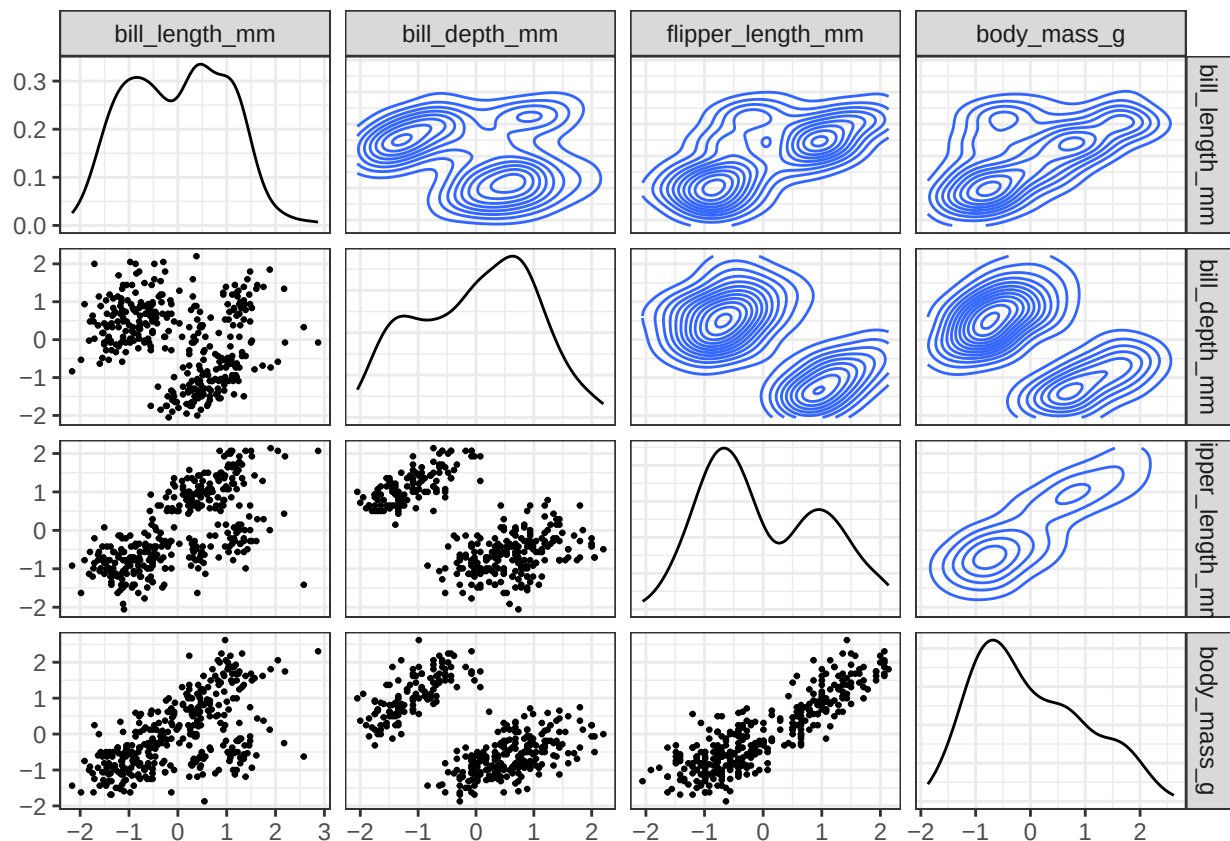
Cluster Dendrogram



```
stats::as.dist(dmat)
stats::hclust(*, "ward.D2")
```

```
###La alternativa K-means
```

```
ggpairs(df1,
  upper = list(continuous = "density"),
  lower = list(continuous = wrap("points", size = 0.5)),
  diag = list(continuous = "densityDiag")) +
  theme_bw()
```



```
set.seed(123)
```

```
dfTask <- makeClusterTask(data = df1)
listLearners("cluster")$class
```

```
## [1] "cluster.cmeans"      "cluster.Cobweb"
## [3] "cluster.dbSCAN"      "cluster.EM"
## [5] "cluster.FarthestFirst" "cluster.kkmeans"
## [7] "cluster.kmeans"      "cluster.MiniBatchKmeans"
## [9] "cluster.SimpleKMeans" "cluster.XMeans"
```

```
kMeans <- makeLearner("cluster.kmeans", par.vals = list(iter.max = 2500, nstart = 25))
kMeans
```

```
## Learner cluster.kmeans from package stats, clue
## Type: cluster
## Name: K-Means; Short name: kmeans
## Class: cluster.kmeans
## Properties: numerics, prob
## Predict-Type: response
## Hyperparameters: centers=2, iter.max=2.5e+03, nstart=25
```

```
kMeansParamSpace <- makeParamSet(
  makeDiscreteParam("centers", values = 2:6),
  makeDiscreteParam("algorithm",
```

```

values = c("Hartigan-Wong", "Lloyd", "MacQueen"))
gridSearch <- makeTuneControlGrid()
kFold <- makeResampleDesc("CV", iters = 25)

set.seed(123)

library(clusterSim)

```

```
## Loading required package: MASS
```

```
##
```

```
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:plotly':
```

```
##
```

```
##      select
```

```
## The following object is masked from 'package:rstatix':
```

```
##
```

```
##      select
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
##      select
```

```

tunedK <- tuneParams(kMeans, task = dfTask,
resampling = kFold,
par.set = kMeansParamSpace,
control = gridSearch,
measures = list(db, G1))

```

```
## [Tune] Started tuning learner cluster.kmeans for parameter set:
```

```

##              Type len Def              Constr Req Tunable Trafo
## centers      discrete - -              2,3,4,5,6 -   TRUE   -
## algorithm discrete - - Hartigan-Wong,Lloyd,MacQueen -   TRUE   -

```

```
## With control class: TuneControlGrid
```

```
## Imputation value: InfImputation value: -0
```

```
## [Tune-x] 1: centers=2; algorithm=Hartigan-Wong
```

```
## [Tune-y] 1: db.test.mean=0.6960118,G1.test.mean=18.9268033; time: 0.0 min
```

```
## [Tune-x] 2: centers=3; algorithm=Hartigan-Wong
```

```
## [Tune-y] 2: db.test.mean=0.7863749,G1.test.mean=18.3047267; time: 0.0 min
```

```

## [Tune-x] 3: centers=4; algorithm=Hartigan-Wong
## [Tune-y] 3: db.test.mean=0.7323548,G1.test.mean=17.1705923; time: 0.0 min
## [Tune-x] 4: centers=5; algorithm=Hartigan-Wong
## [Tune-y] 4: db.test.mean=0.7201345,G1.test.mean=17.9362909; time: 0.0 min
## [Tune-x] 5: centers=6; algorithm=Hartigan-Wong
## [Tune-y] 5: db.test.mean=0.6519229,G1.test.mean=18.2417129; time: 0.0 min
## [Tune-x] 6: centers=2; algorithm=Lloyd
## [Tune-y] 6: db.test.mean=0.6960118,G1.test.mean=18.9268033; time: 0.0 min
## [Tune-x] 7: centers=3; algorithm=Lloyd
## [Tune-y] 7: db.test.mean=0.7863749,G1.test.mean=18.3047267; time: 0.0 min
## [Tune-x] 8: centers=4; algorithm=Lloyd
## [Tune-y] 8: db.test.mean=0.7323548,G1.test.mean=17.1705923; time: 0.0 min
## [Tune-x] 9: centers=5; algorithm=Lloyd
## [Tune-y] 9: db.test.mean=0.7201345,G1.test.mean=17.9362909; time: 0.0 min
## [Tune-x] 10: centers=6; algorithm=Lloyd
## [Tune-y] 10: db.test.mean=0.6483427,G1.test.mean=18.2799837; time: 0.0 min
## [Tune-x] 11: centers=2; algorithm=MacQueen
## [Tune-y] 11: db.test.mean=0.6960118,G1.test.mean=18.9268033; time: 0.0 min
## [Tune-x] 12: centers=3; algorithm=MacQueen
## [Tune-y] 12: db.test.mean=0.7863749,G1.test.mean=18.3047267; time: 0.0 min
## [Tune-x] 13: centers=4; algorithm=MacQueen
## [Tune-y] 13: db.test.mean=0.7323548,G1.test.mean=17.1705923; time: 0.0 min
## [Tune-x] 14: centers=5; algorithm=MacQueen
## [Tune-y] 14: db.test.mean=0.7201345,G1.test.mean=17.9362909; time: 0.0 min
## [Tune-x] 15: centers=6; algorithm=MacQueen
## [Tune-y] 15: db.test.mean=0.6468298,G1.test.mean=18.3416687; time: 0.0 min
## [Tune] Result: centers=6; algorithm=MacQueen : db.test.mean=0.6468298,G1.test.mean=18.3416687

```

```
tunedK
```

```
## Tune result:  
## Op. pars: centers=6; algorithm=MacQueen  
## db.test.mean=0.6468298,G1.test.mean=18.3416687
```

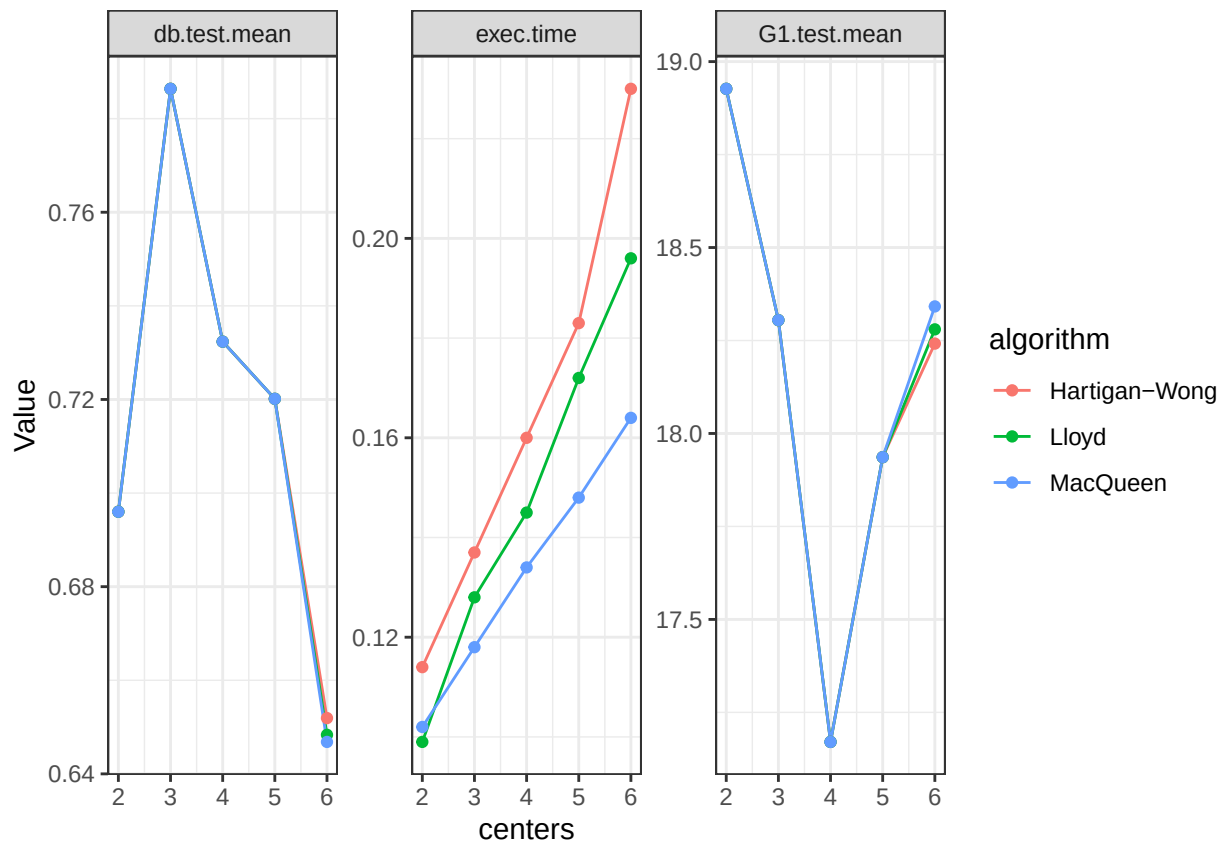
```
set.seed(123)
```

```
kMeansTuningData <- generateHyperParsEffectData(tunedK)  
kMeansTuningData$data
```

##	centers	algorithm	db.test.mean	G1.test.mean	iteration	exec.time
## 1	2	Hartigan-Wong	0.6960118	18.92680	1	0.114
## 2	3	Hartigan-Wong	0.7863749	18.30473	2	0.137
## 3	4	Hartigan-Wong	0.7323548	17.17059	3	0.160
## 4	5	Hartigan-Wong	0.7201345	17.93629	4	0.183
## 5	6	Hartigan-Wong	0.6519229	18.24171	5	0.230
## 6	2	Lloyd	0.6960118	18.92680	6	0.099
## 7	3	Lloyd	0.7863749	18.30473	7	0.128
## 8	4	Lloyd	0.7323548	17.17059	8	0.145
## 9	5	Lloyd	0.7201345	17.93629	9	0.172
## 10	6	Lloyd	0.6483427	18.27998	10	0.196
## 11	2	MacQueen	0.6960118	18.92680	11	0.102
## 12	3	MacQueen	0.7863749	18.30473	12	0.118
## 13	4	MacQueen	0.7323548	17.17059	13	0.134
## 14	5	MacQueen	0.7201345	17.93629	14	0.148
## 15	6	MacQueen	0.6468298	18.34167	15	0.164

```
gatheredTuningData <- gather(kMeansTuningData$data,  
key = "Metric",  
value = "Value",  
c(-centers, -iteration, -algorithm))
```

```
ggplot(gatheredTuningData, aes(centers, Value, col = algorithm)) +  
facet_wrap(~ Metric, scales = "free_y") +  
geom_line() +  
geom_point() +  
theme_bw()
```



```
set.seed(123)

tunedKMeans <- setHyperPars(kMeans, par.vals = tunedK$x)
tunedKMeansModel <- train(tunedKMeans, dfTask)
kMeansModelData <- getLearnerModel(tunedKMeansModel)
kMeansModelData$iter

## [1] 5

tunedKMeans

## Learner cluster.kmeans from package stats,clue
## Type: cluster
## Name: K-Means; Short name: kmeans
## Class: cluster.kmeans
## Properties: numerics,prob
## Predict-Type: response
## Hyperparameters: centers=6,iter.max=2.5e+03,nstart=25,algorithm=MacQueen

df2 <- mutate(df1,kMeansCluster = as.factor(kMeansModelData$cluster))

table(df2$kMeansCluster)

##
## 1 2 3 4 5 6
## 33 32 57 61 93 66
```



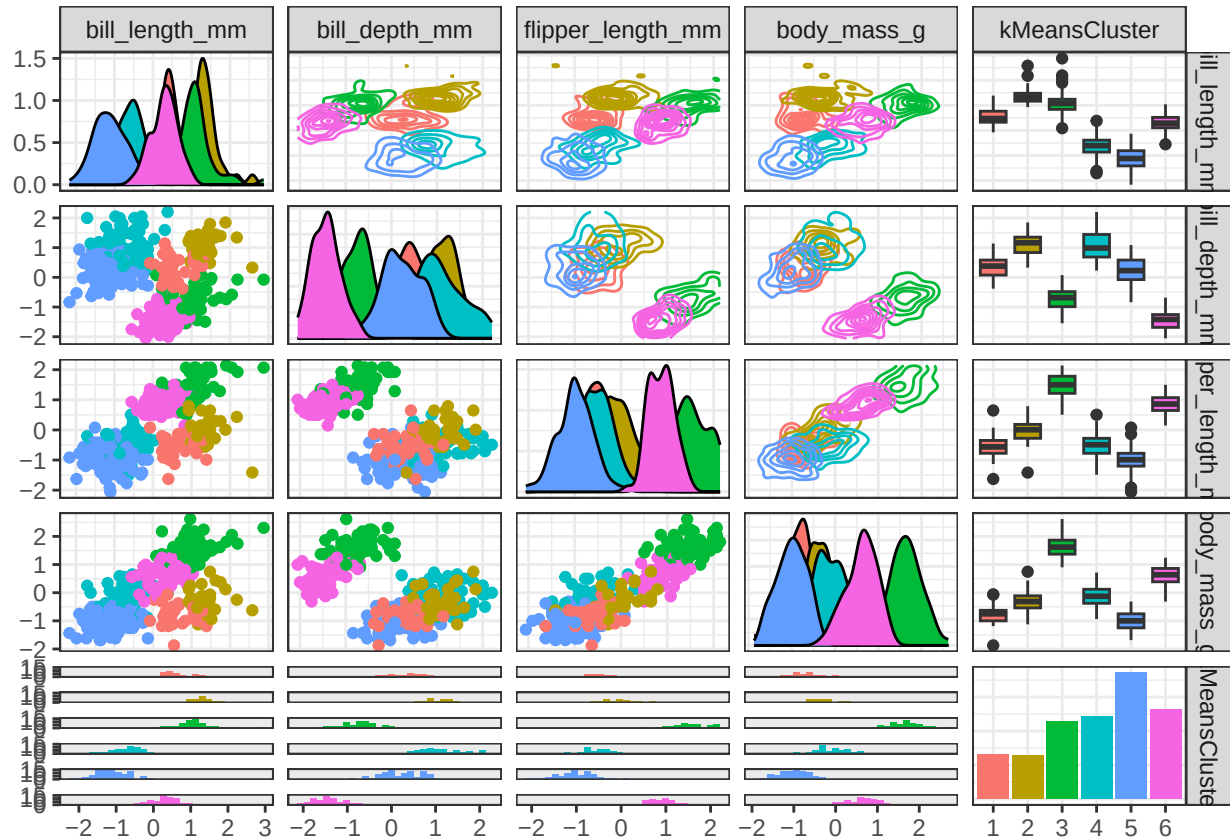
```
ggpairs(df2, aes(col = kMeansCluster), upper = list(continuous = "density")) +  
theme_bw()
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



```
newData <- df2[32:35, 1:ncol(df1)]
```

```
predict(tunedKMeansModel, newdata = newData)
```

```
## Prediction: 4 observations
```

```
## predict.type: response
```

```
## threshold:
```

```
## time: 0.00
```

```
## response
```

```
## 32      5
```

```
## 33      4
```

```
## 34      5
```

```
## 35      4
```

```
###¿2 clusters?
```

```
###Mas tecnicas para determinar el numero de clusters subyacente a estos datos
```

```
set.seed(123)
```

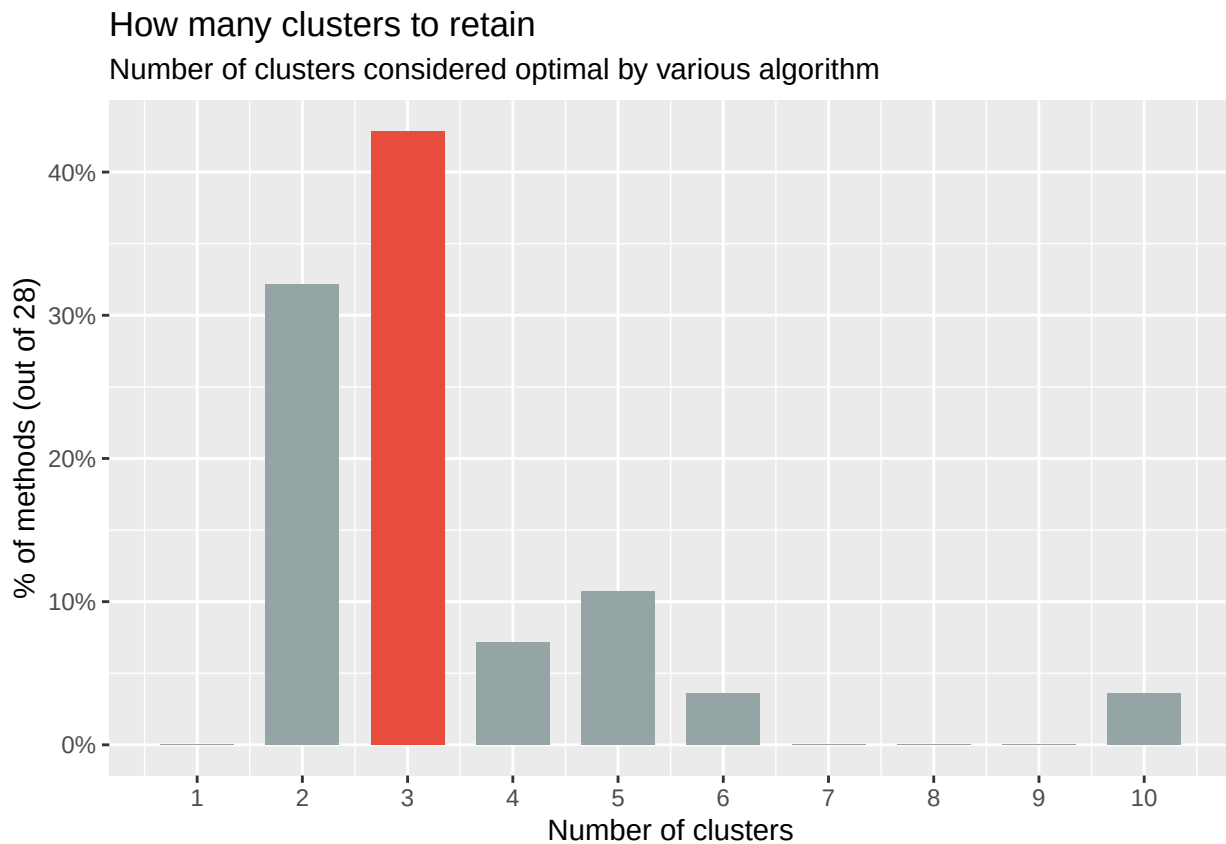
```
nc <- n_clusters(  
  df1,  
  standardize = FALSE,  
  include_factors = FALSE,  
  package = c("easystats", "NbClust", "mclust"),  
  fast = TRUE,  
  nbclust_method = "kmeans",  
  n_max = 10  
)  
nc
```

```
## # Method Agreement Procedure:
```

```
##
```

```
## The choice of 3 clusters is supported by 12 (42.86%) methods out of 28 (Hartigan, Scott, Marriot, tr
```

```
plot(nc)
```



```
###

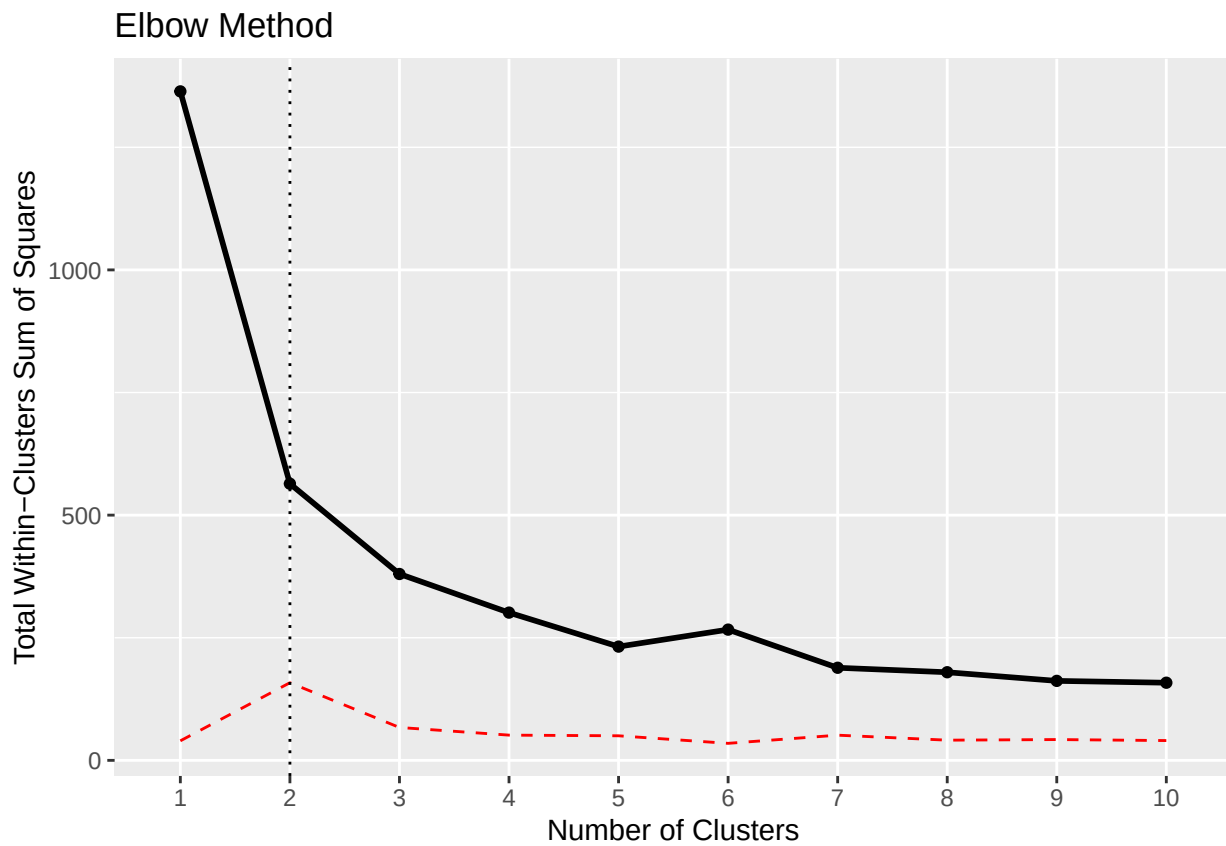
set.seed(123)

elbow<-n_clusters_elbow(
  df1,
  standardize = TRUE,
  include_factors = FALSE,
  clustering_function = stats::kmeans,
  n_max = 10
)

elbow
```

The Elbow method, that aims at minimizing the total intra-cluster variation (i.e., the total within-

```
plot(elbow)
```



```
###

set.seed(123)

gap<-n_clusters_gap(
  df1,
  standardize = TRUE,
```

```

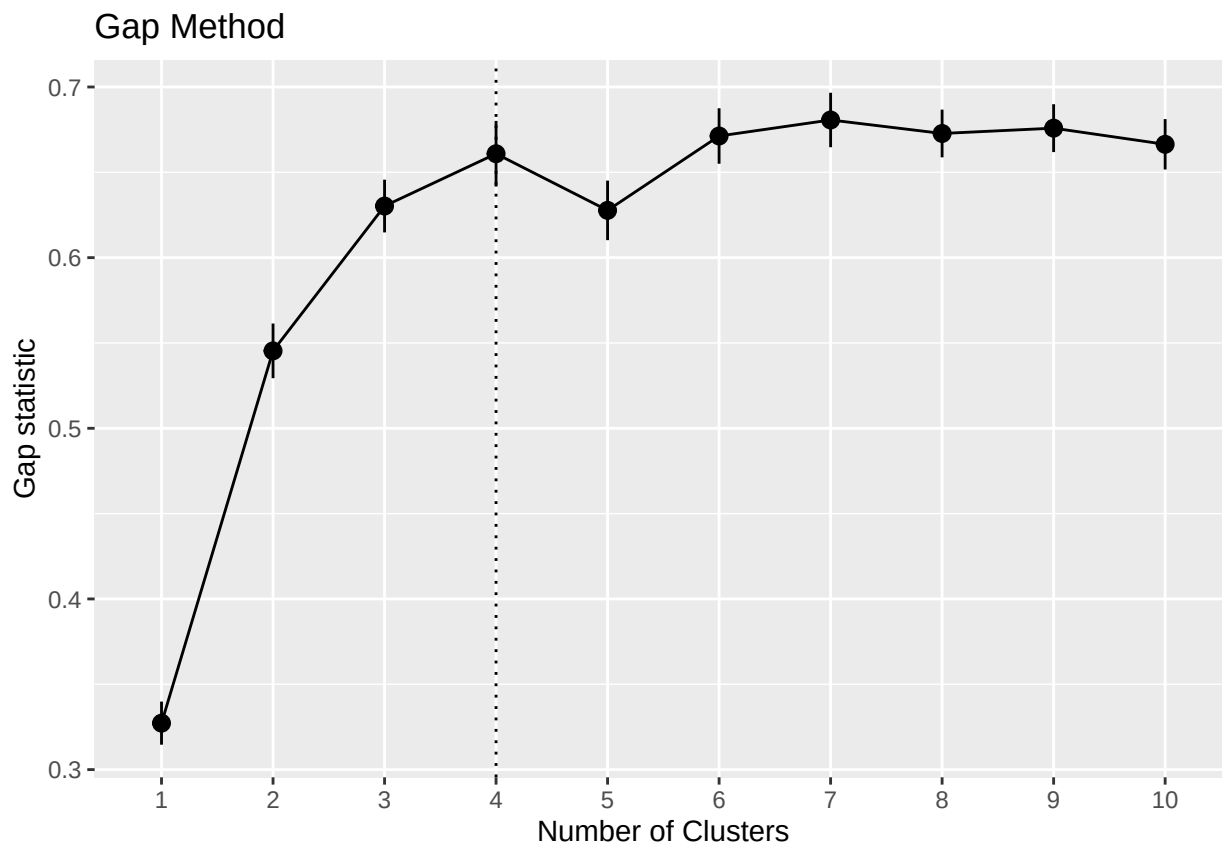
include_factors = FALSE,
clustering_function = stats::kmeans,
n_max = 10,
gap_method = "firstSEmax"
)

```

```
gap
```

The Gap method, that compares the total intracluster variation of k clusters with their expected value

```
plot(gap)
```



```

###

set.seed(123)

n_clusters_silhouette(
  df1,
  standardize = TRUE,
  include_factors = FALSE,
  clustering_function = stats::kmeans,
  n_max = 10
)

```

The Silhouette method, based on the average quality of clustering, suggests that the optimal number of

```
###

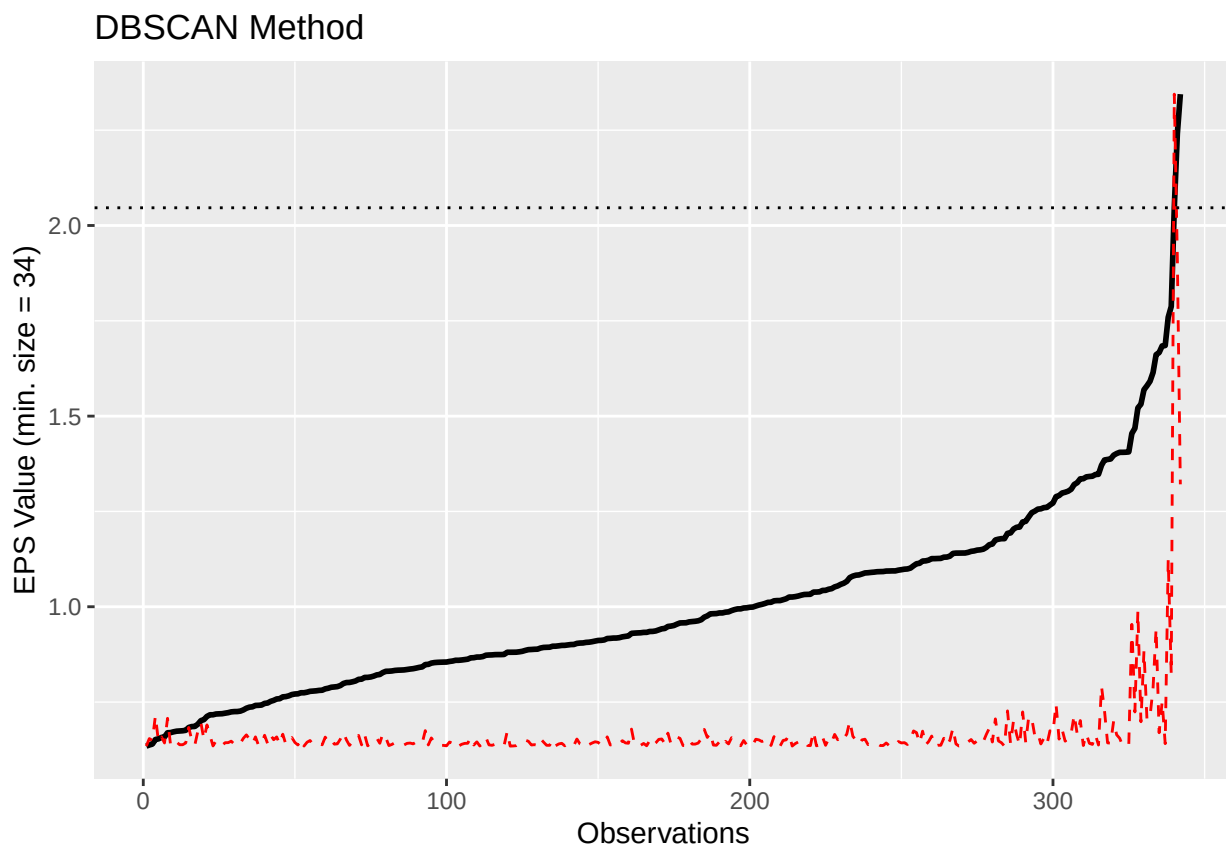
set.seed(123)

dbscan<-n_clusters_dbscan(
  df1,
  standardize = TRUE,
  include_factors = FALSE,
  method = c("kNN", "SS"),
  min_size = 0.1,
  eps_n = 50,
  eps_range = c(0.1, 3)
)

dbscan
```

The DBSCAN method, based on the total clusters sum of squares, suggests that the optimal $\text{eps} = 2.046$

```
plot(dbscan)
```



```
###

set.seed(123)

hclust<-n_clusters_hclust(
```

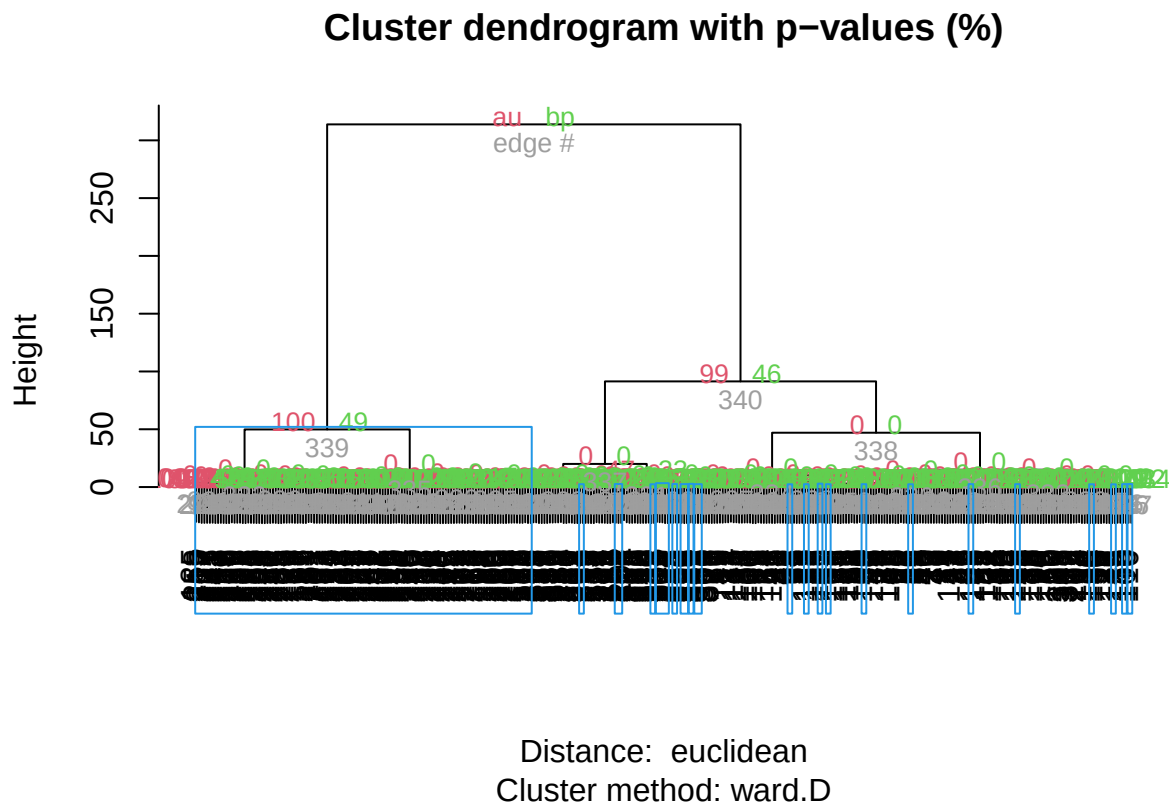
```

df1,
  standardize = TRUE,
  include_factors = FALSE,
  distance_method = "euclidean",
  hclust_method = "ward.D",
  ci = 0.95,
  iterations = 20
)
hclust

```

The bootstrap analysis of hierarchical clustering highlighted 21 significant clusters.

```
plot(hclust)
```



```

###
set.seed(123)

hopkins_stat_value <- hopkins(df1, m = 20)
hopkins_stat_value

```

```
## [1] 0.9966263
```

```
clustertend::hopkins(df1)
```

```
## $H
```

```
## [1] 0.2220803
```